

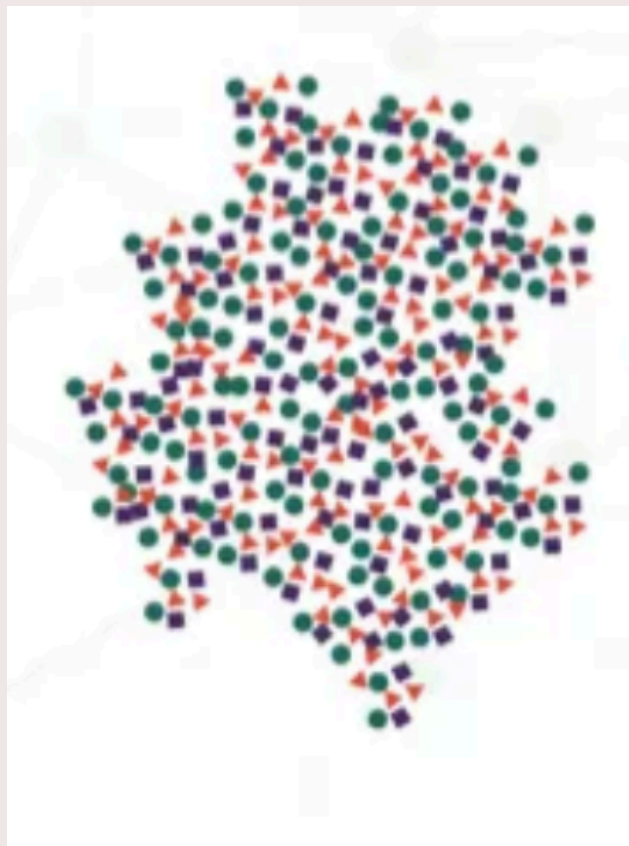
# **Supervised learning : Artificial Neural Networks**

Learning patterns from the data

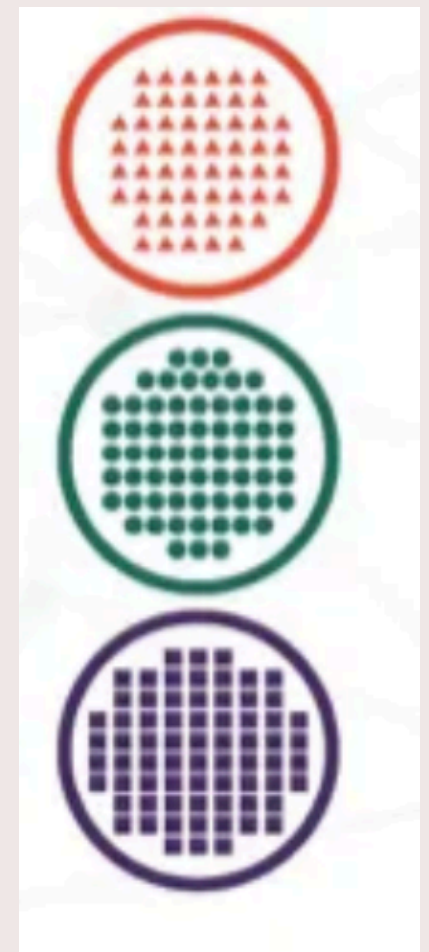
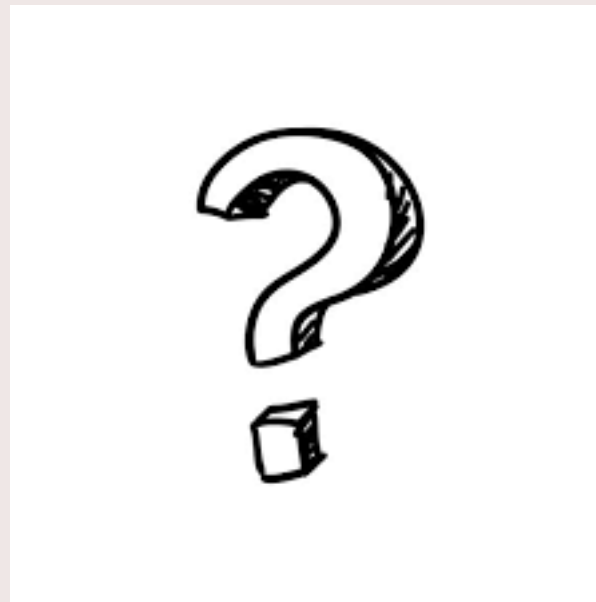
**Dr Patricia Ryser-Welch, October 2025**

# A simple classification problem?

How can I predict the each ball is either red, green, or purple?

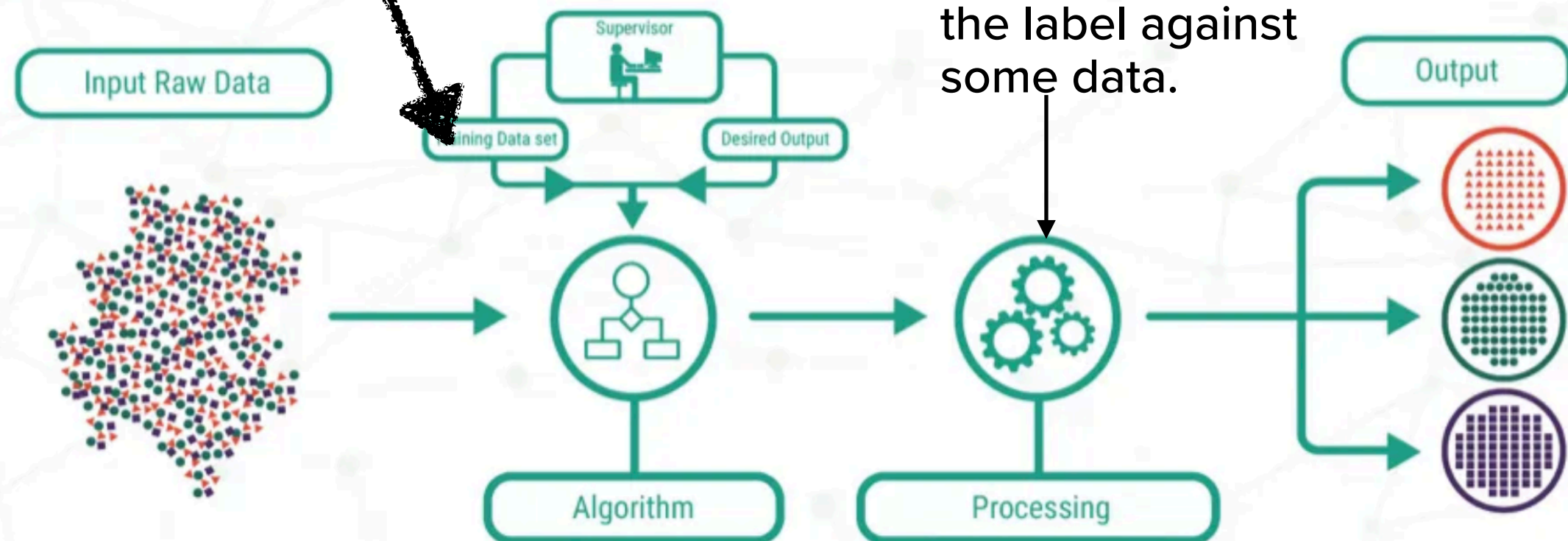


Are there any patterns we can learn from the data?



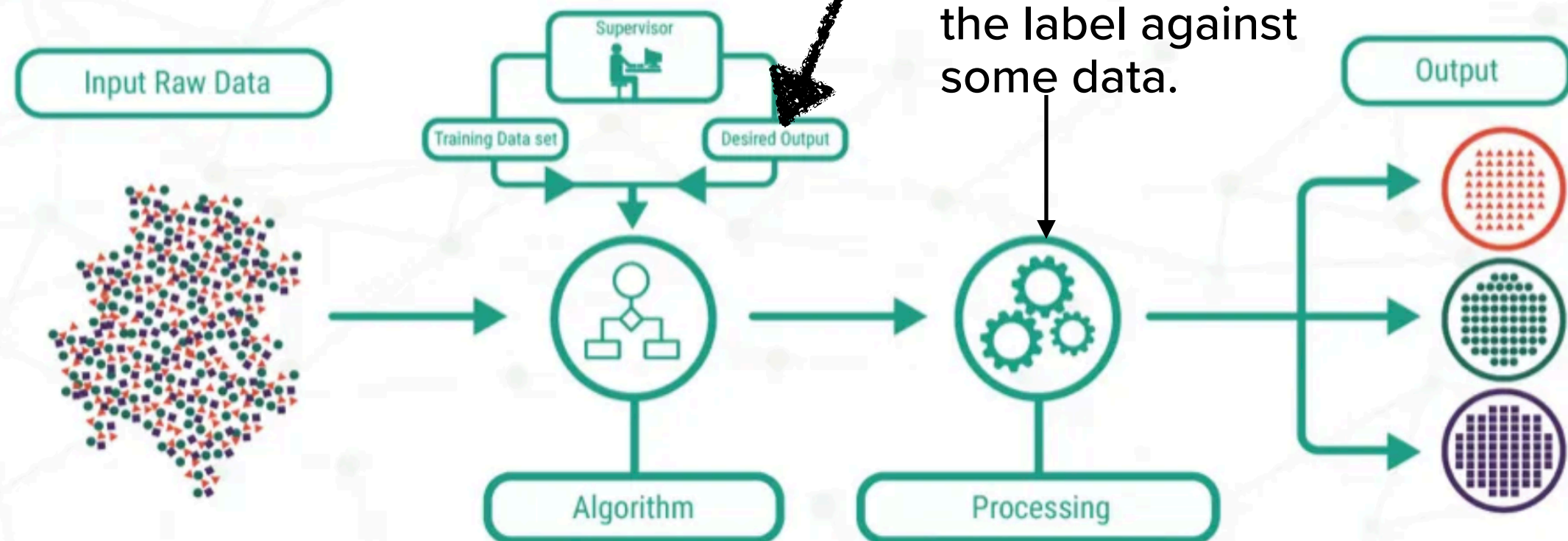
A training and validation datasets are both data that represents the features expected in the data.

# SUPERVISED LEARNING



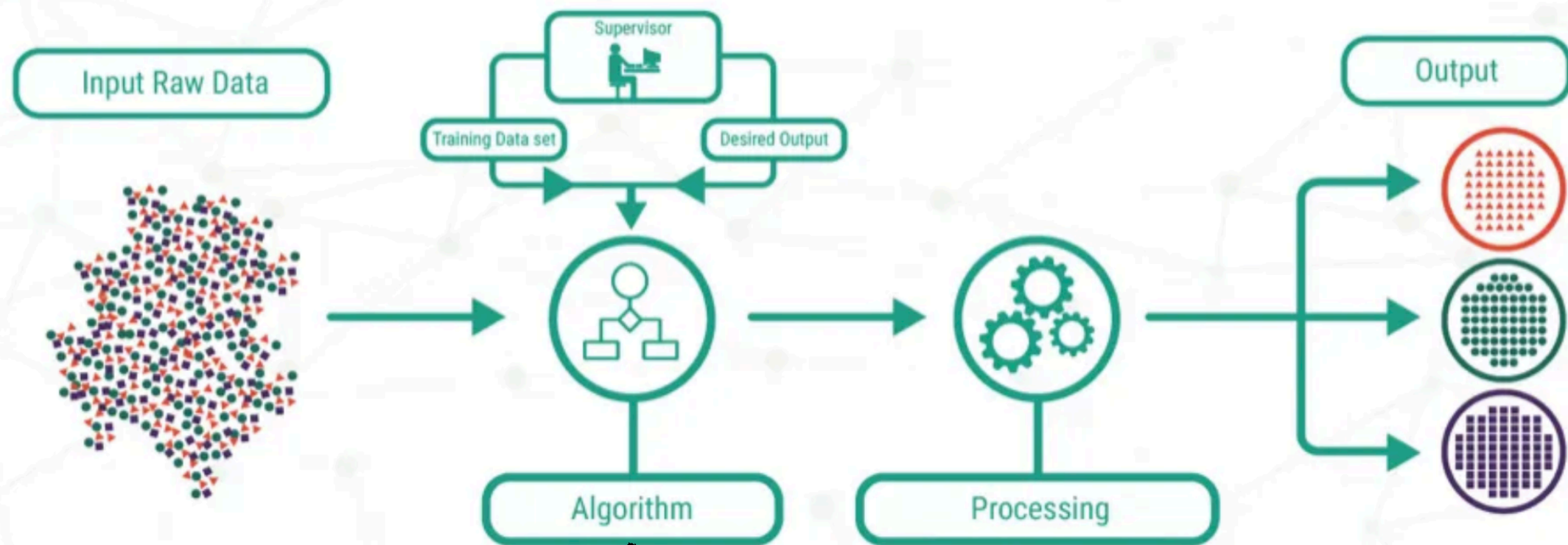
The labelled data indicates the desired output!

# SUPERVISED LEARNING



Methodologies used to predict the label against some data.

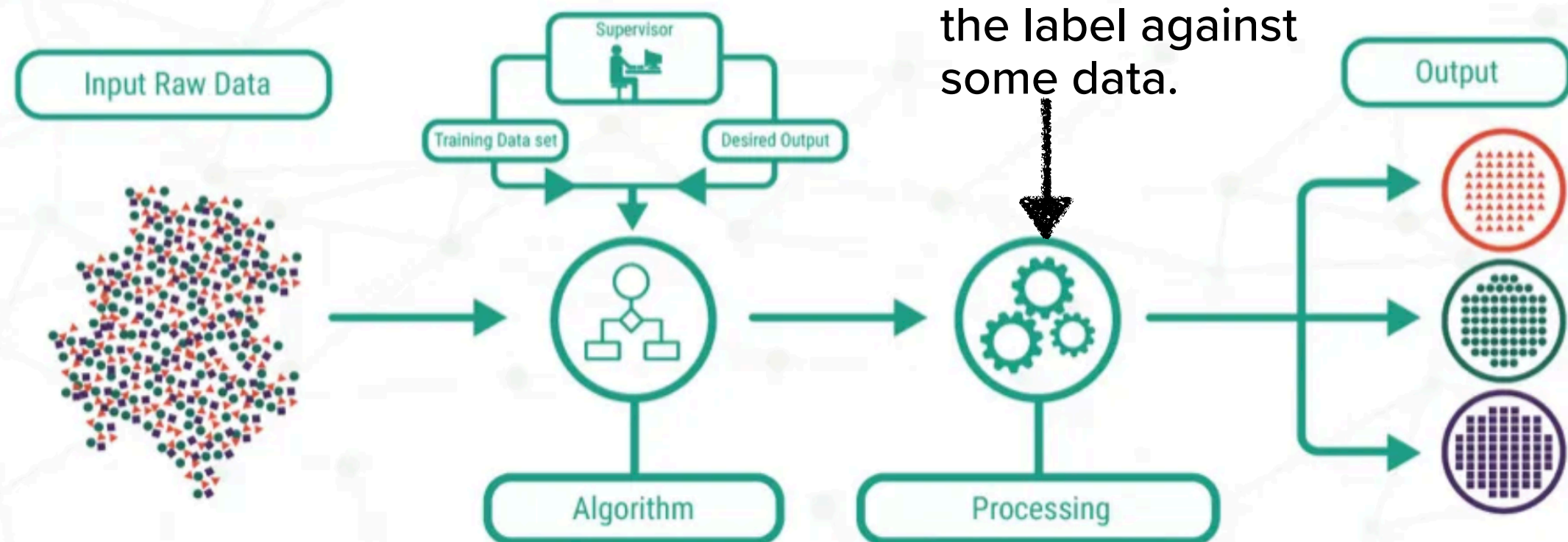
# SUPERVISED LEARNING



A process is used to find a methodologies/processing that can transform some data against the labels.

# SUPERVISED LEARNING

Methodologies used to predict the label against some data.



**Now I have a processing method to classify red, green, and purple beads, what would happen if some beads are yellow?**

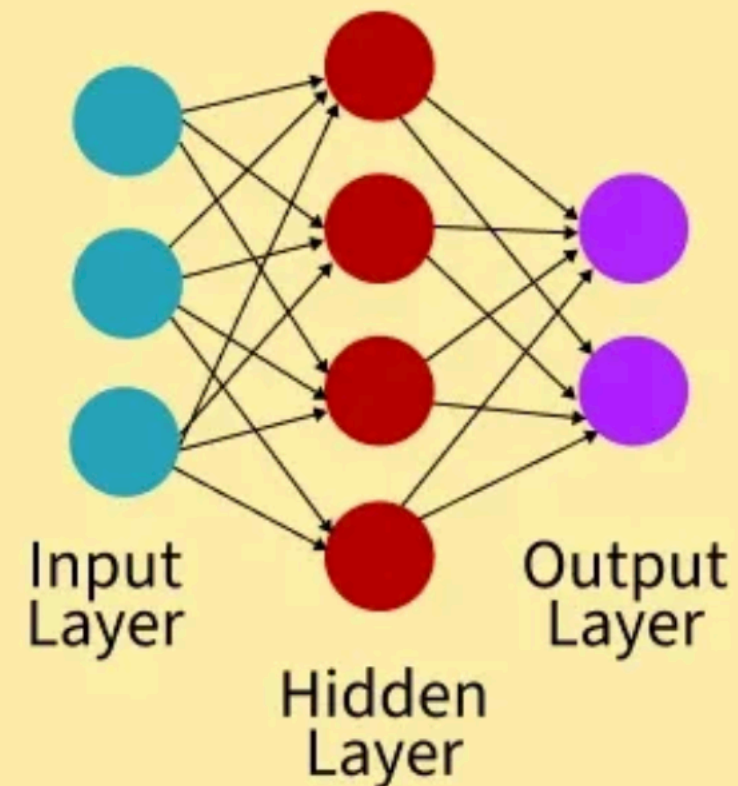
**What would happen if I have more than one feature for classifying?**

**The shape, the width, depth, length, radius**

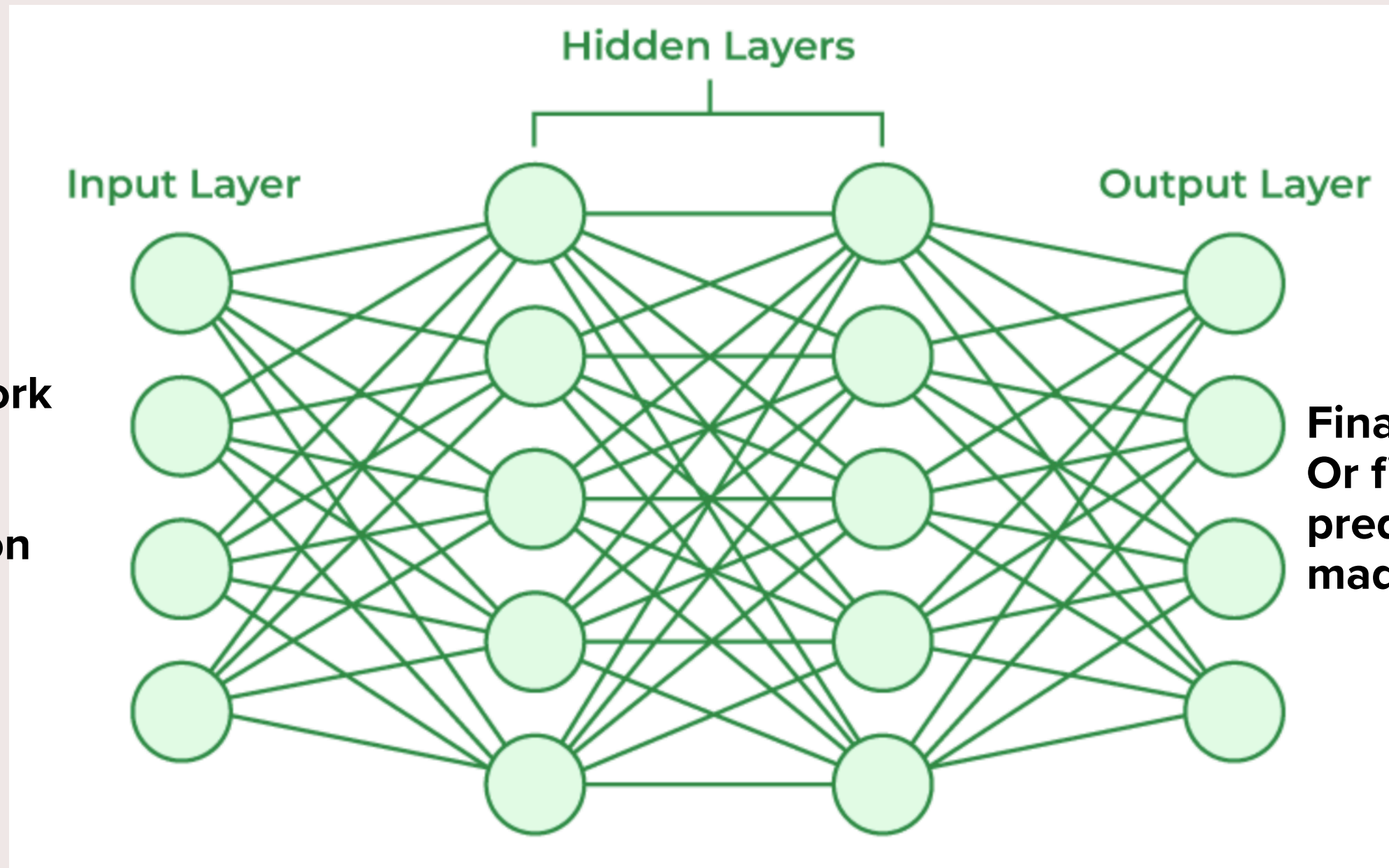
# One method: Artificial Neural Network

## What Are Artificial Neural Networks?

- Simulate the human brain's information processing.
- Consist of input, hidden and output layers.
- Learn patterns and make predictions from data.



# ANN - Architecture is very complex



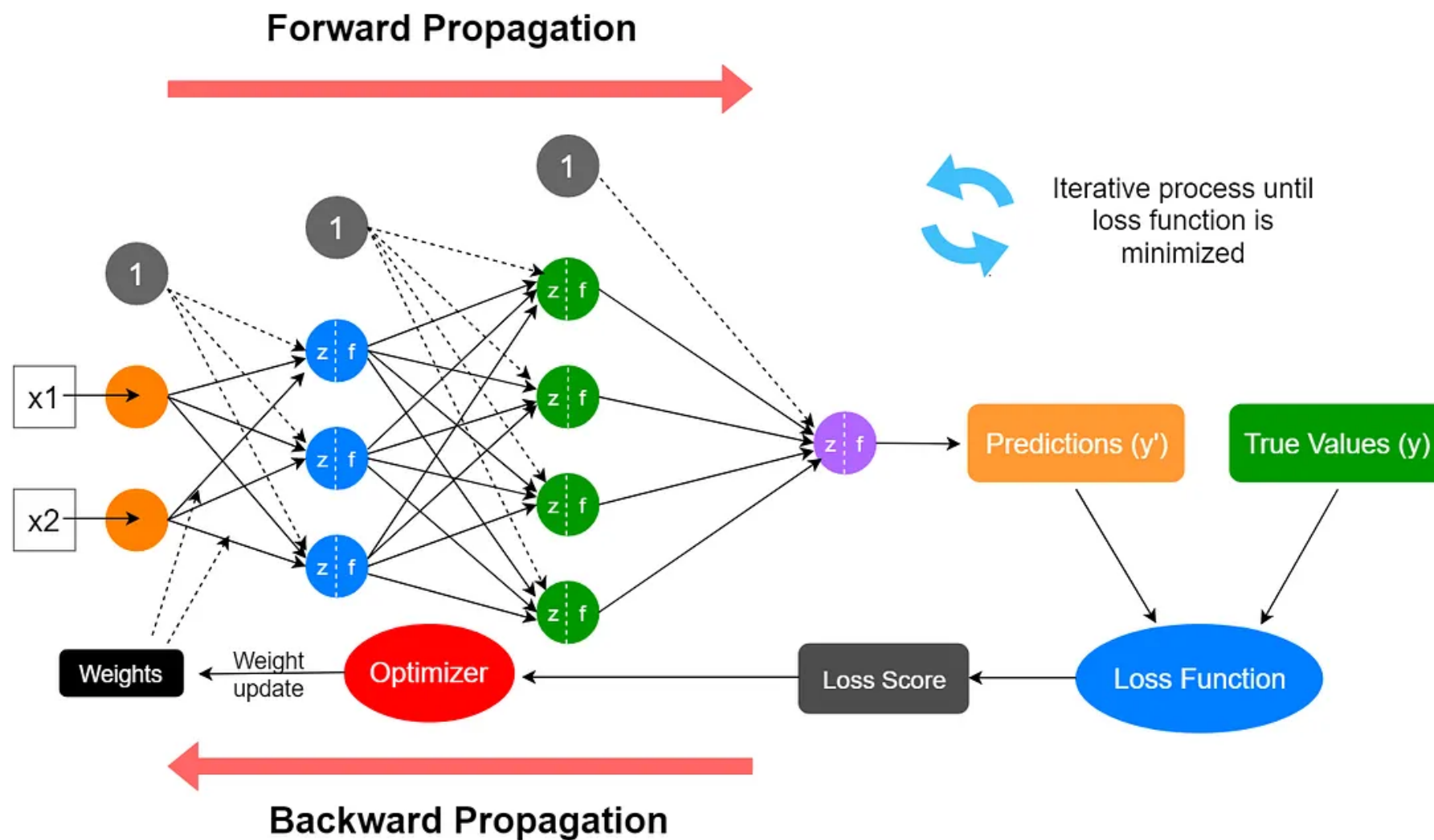
**The network  
receives  
some  
information**

**Final decision  
Or final  
prediction is  
made**

**Transforms data into more abstract information**

# Learning pattern within some given data

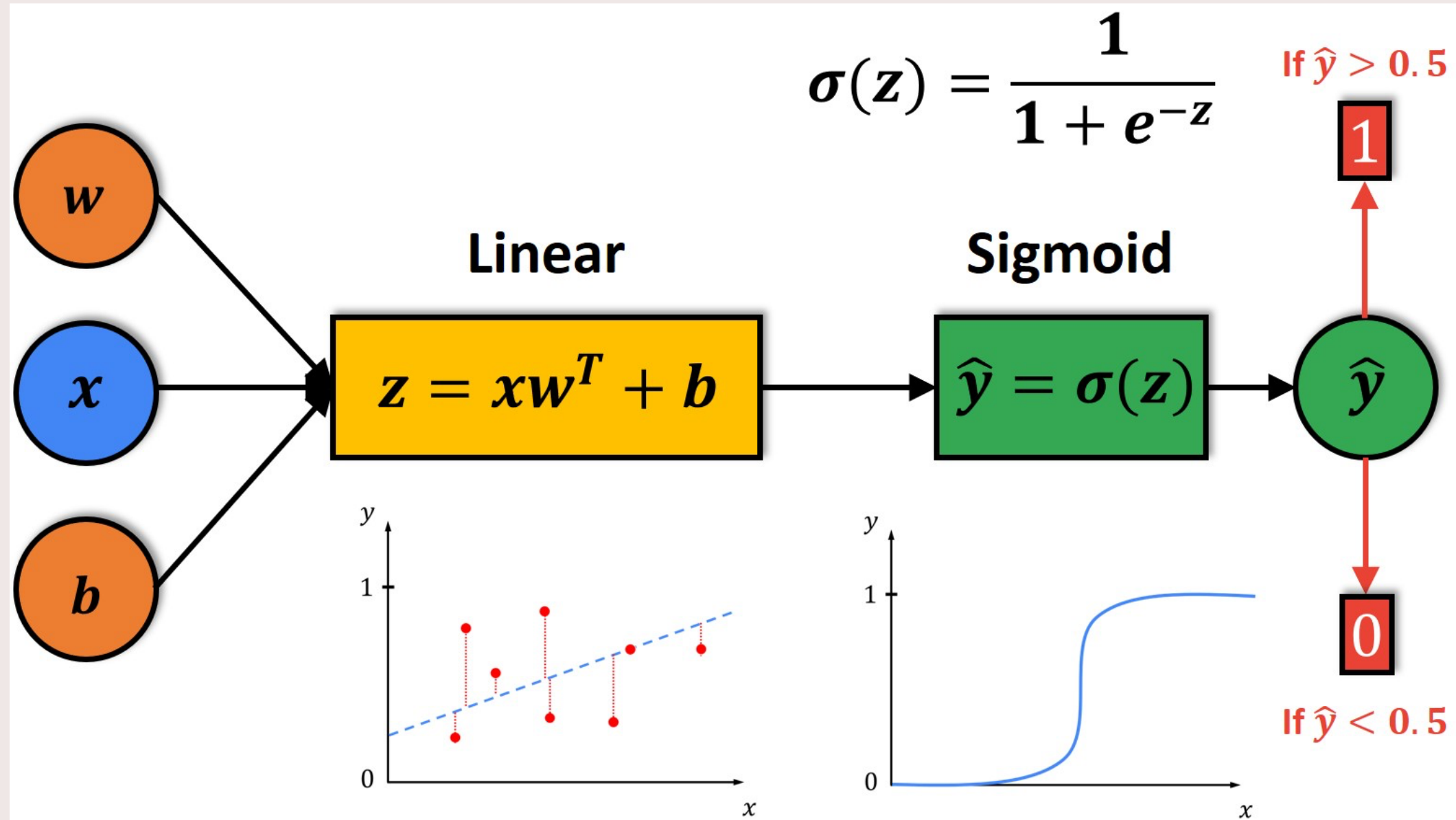
Model fitting or learning - An epoch



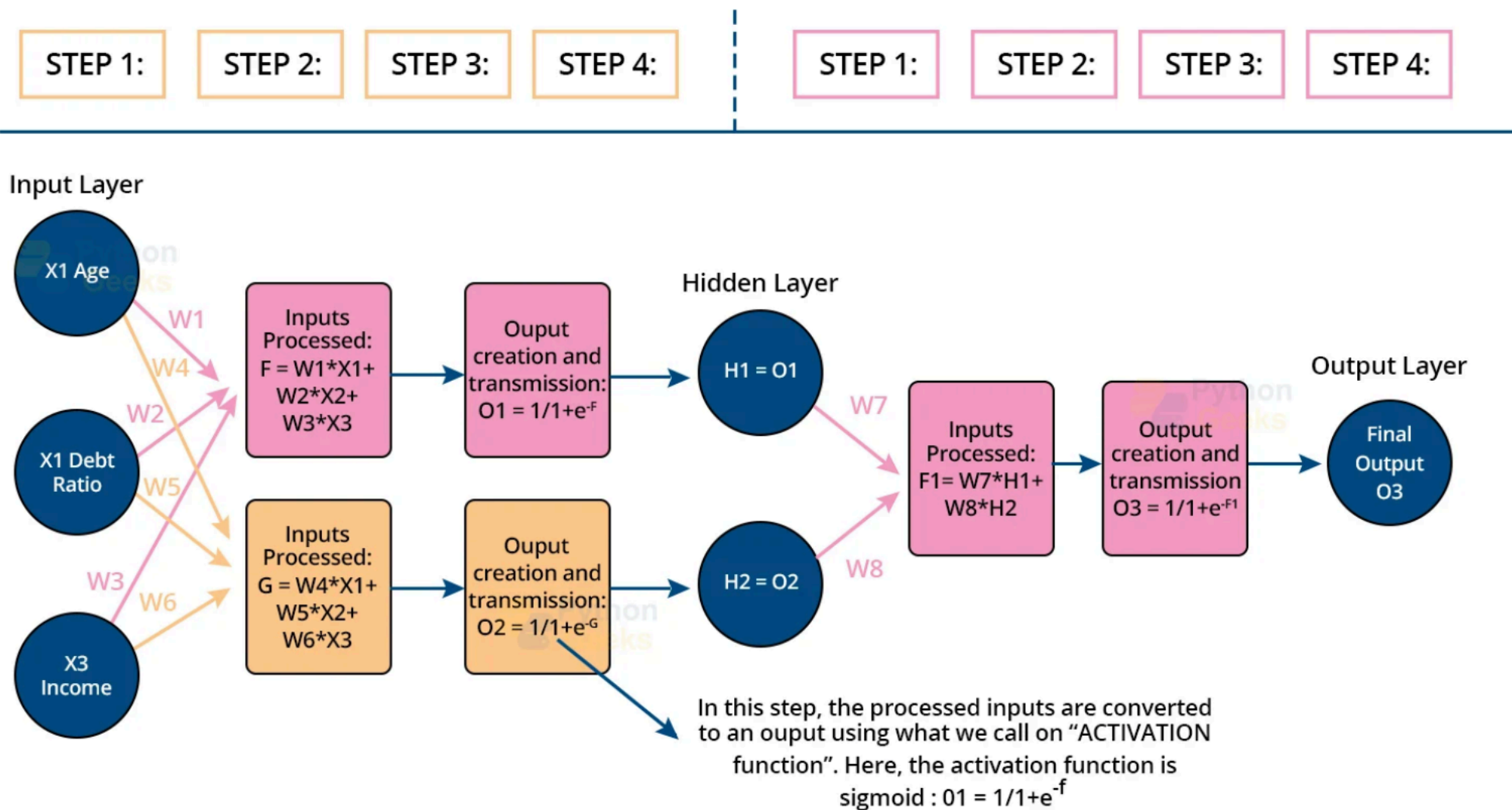
<https://medium.com/data-science-365/overview-of-a-neural-networks-learning-process-61690a502fa>

# Forward propagation : inside a neuron ...

A logistic regression

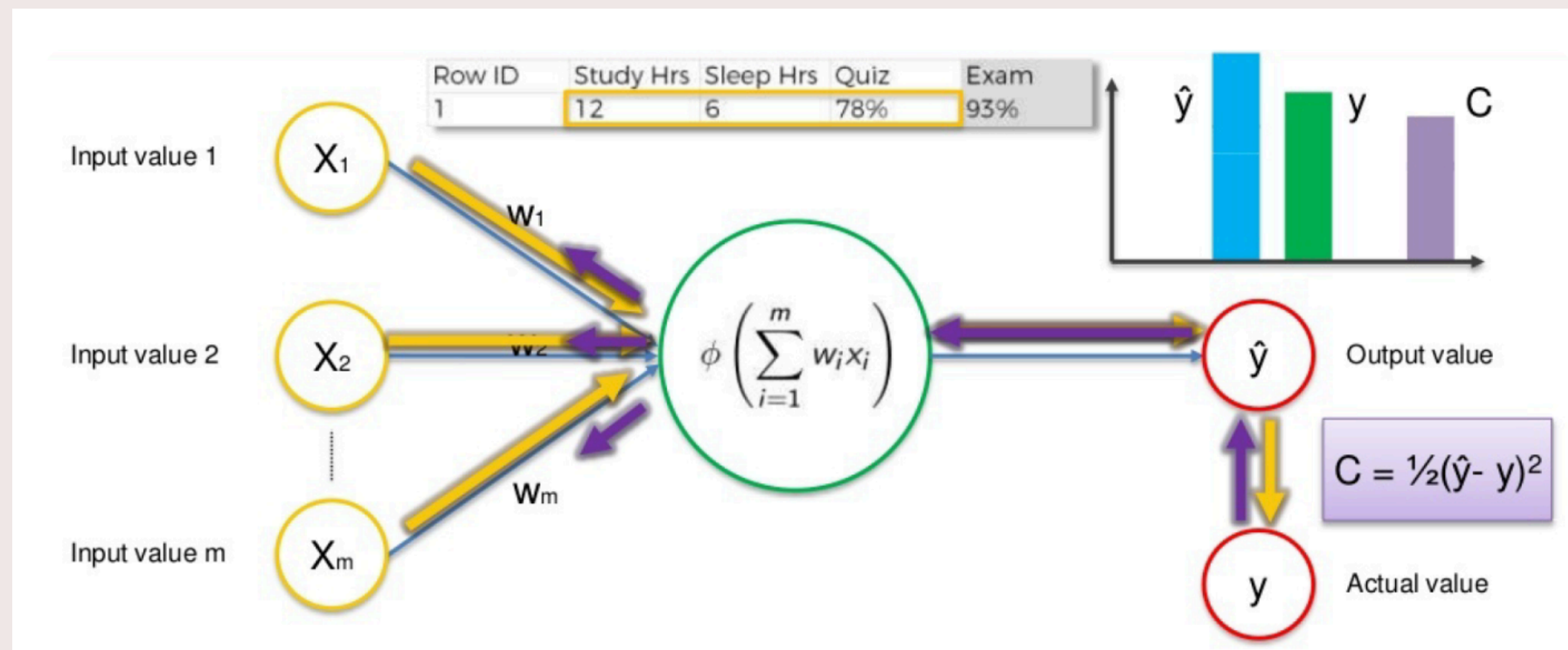


# Something a bit more complex ....

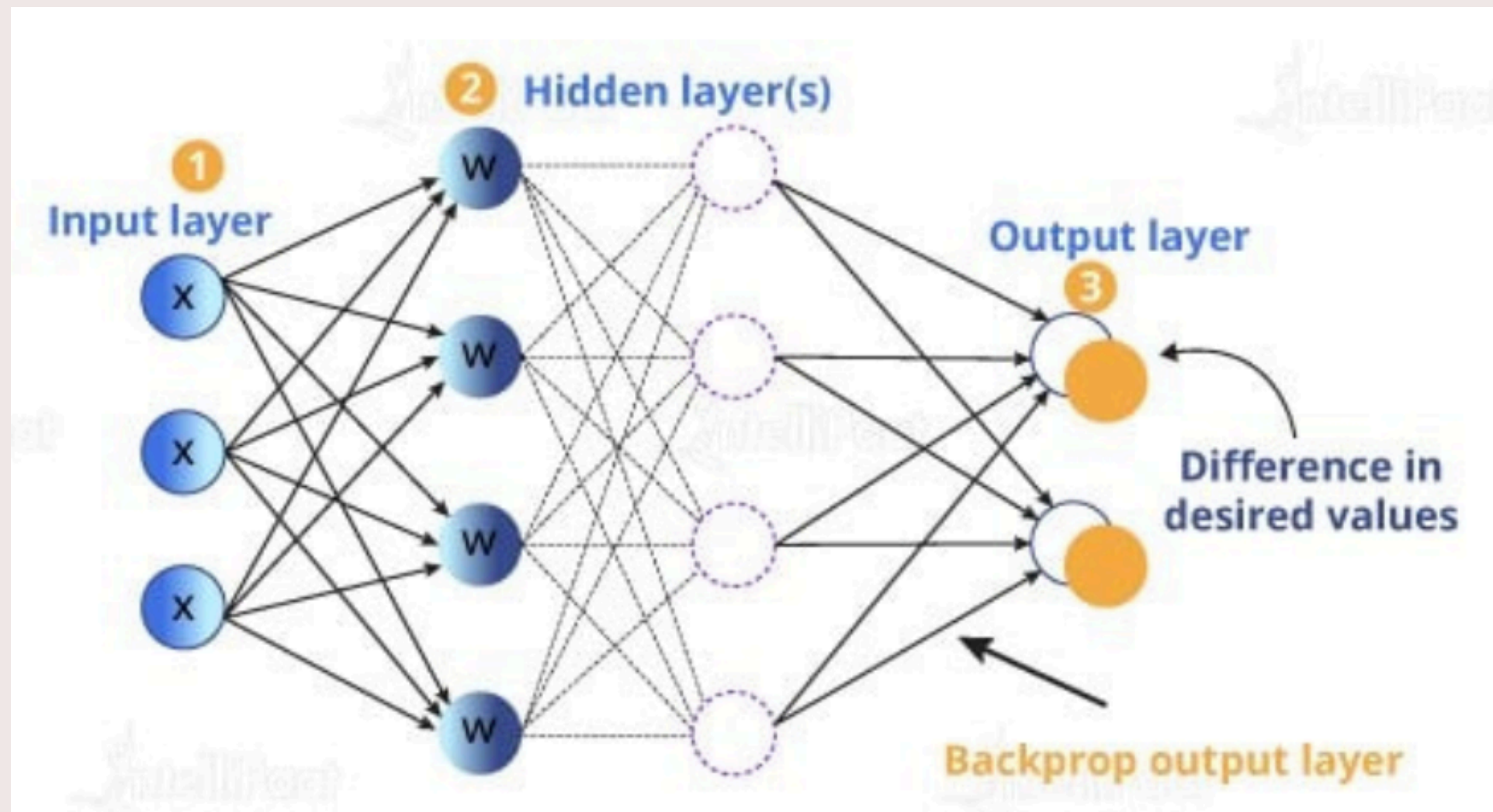


# Back-propagation a single neuron

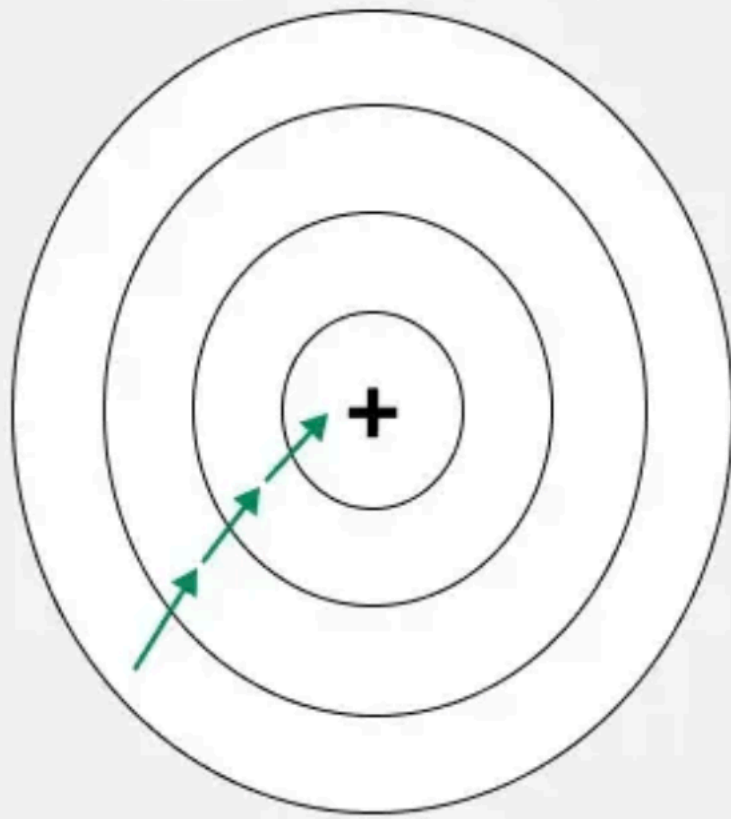
Correction of the weights



# Back-propagation



## Gradient Descent



# Learning rate

Controlling the descent to the most optimum model

$$w = w - \alpha \cdot \nabla L(w)$$

Where:

- $w$  represents the weights
- $\alpha$  is the learning rate
- $\nabla L(w)$  is the gradient of the loss function

# Impact of learning rate

## **Low Learning Rate:**

- Leads to slow convergence
- Requires more training epochs
- Can improve accuracy but increases computation time

## **High Learning Rate:**

- Speeds up training
- Risks of overshooting optimal weights
- May cause instability or divergence of the loss function

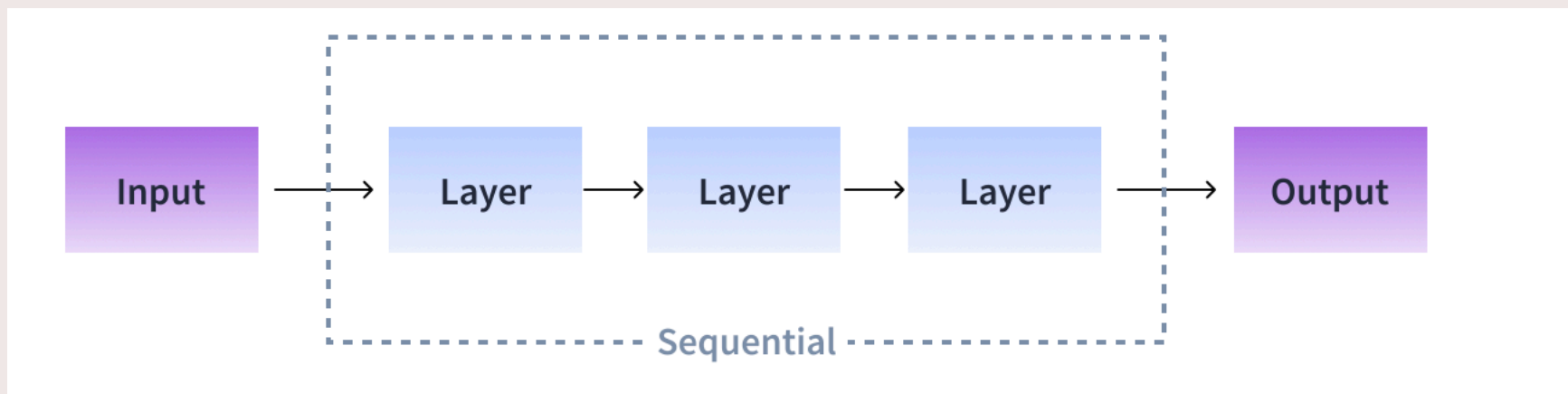
## **Optimal Learning Rate:**

- Balances training speed and model accuracy
- Ensures stable convergence without excessive training time

<https://www.dataquest.io/blog/tutorial-introduction-to-keras/>

# Training with Keras....

## Defining a Neural network



```
from tf.keras import Sequential  
  
model = Sequential()
```

```
from tf.keras import Dense  
  
model.add(Dense(input_shape=(16,)))  
model.add(Dense(8))  
model.add(Dense(4))  
model.add(Dense(2))  
model.add(Dense(1))
```

[https://keras.io/api/layers/core\\_layers/dense/](https://keras.io/api/layers/core_layers/dense/)

# Compiling the model

## Compiling a Model with Keras

Once you've built a model using the Keras Sequential API, you'll need to compile it before it can be used for training. When compiling the model, you'll need to specify both a loss function and an optimizer. There are many different options to choose from:

### Loss functions:

- *Mean Squared Error (MSE)*: a common loss function for regression problems — measures the average squared difference between the predicted and actual values.
- *Binary Crossentropy*: a loss function for binary classification problems — measures the cross-entropy between the predicted and actual binary distributions.
- *Categorical Crossentropy*: a loss function for multi-class classification problems — measures the cross-entropy between the predicted and actual categorical distributions.

### Optimizers:

- *Stochastic Gradient Descent (SGD)*: a simple optimization algorithm that updates the parameters by computing the gradient of the loss function with respect to the parameters.
- *Adam*: an optimization algorithm that adapts the learning rate based on the historical gradient information.
- *RMSprop*: an optimization algorithm that uses a moving average of squared gradients to normalize the gradient updates.

```
model.compile(loss='mean_squared_error', optimizer='adam')
```

# Model training (model fitting)

## Fitting a Model with Keras

Finally, to use Keras for deep learning, the compiled model must be **fit** to a training dataset. The training dataset should be prepared using a process that separates the independent variables, the features (or **X** variable) from the dependent variable, the target (or **y** variable). These variables should also be converted to NumPy arrays and reshaped as needed, and the **input\_shape** parameter of the neural network will need to be configured to match the training data shape. Once all of these preprocessing steps are in place, you can simply fit the model to the training data like so:

```
model.fit(X_train, y_train)
```

To evaluate the performance of the model after training, you can use **evaluate**:

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
```

<https://www.dataquest.io/blog/tutorial-introduction-to-keras/>